Lecture 20

More Dynamic Programming Examples

Source: Introduction to Algorithms, CLRS

Subsequence

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements dropped.

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements dropped.

Example:

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements dropped.

Example: Let $S =$ "abrakadabra" be a sequence.

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements dropped.

Example: Let $S =$ "abrakadabra" be a sequence.

Some subsequence of S are : "arkdra"

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements dropped.

Example: Let $S =$ "abrakadabra" be a sequence.

Some subsequence of S are : "arkdra", "a"

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements dropped.

Example: Let $S =$ "abrakadabra" be a sequence.

Some subsequence of S are : "arkdra", "a", "aaaaa"

Subsequence

Defn: A subsequence of a given sequence is just the given sequence with 0 or more elements dropped.

Example: Let $S =$ "abrakadabra" be a sequence.

Some subsequence of S are : "arkdra", "a", "aaaaa", "abrakadabra".

Longest Common Subsequence

Longest Common Subsequence

LCS:

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Example: $X =$ "iitjodhpur", $Y =$ "iitindore"

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Example: $X =$ "iitjodhpur", $Y =$ "iitindore"

Some common sequences of X and Y are: "iit", "dr", "tor".

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Example: $X =$ "iitjodhpur", $Y =$ "iitindore"

Some common sequences of X and Y are: "iit", "dr", "tor".

Some longest common sequences of X and Y are: "iitor", "iitdr".

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Application of LCS:

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Application of LCS:

- LCS of two sequences or strings is a measure of how similar they are.

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Application of LCS:

- LCS of two sequences or strings is a measure of how similar they are.
- Used to find similarity of DNAs which can be seen as strings of “A”, “C”, “G”, and “T”

Longest Common Subsequence

LCS:

Input: Two sequences X and Y.

Output: Length of the longest common subsequence.

Application of LCS:

- LCS of two sequences or strings is a measure of how similar they are.
- Used to find similarity of DNAs which can be seen as strings of “A”, “C”, “G”, and “T” characters which represent nucleotides.

Brute Force for LCS

Brute Force for LCS

BruteForceLCS(X, Y):

Brute Force for LCS

BruteForceLCS(X, Y):

1. $lcs = 0$

Brute Force for LCS

BruteForceLCS(X, Y):

1. $lcs = 0$
2. **for** every subsequence x of X

Brute Force for LCS

BruteForceLCS(X, Y):

1. $lcs = 0$
2. **for** every subsequence x of X
3. **if** x is a subsequence of Y

Brute Force for LCS

BruteForceLCS(X, Y):

1. $lcs = 0$
2. **for** every subsequence x of X
3. **if** x is a subsequence of Y
4. $lcs = \textbf{Max}(lcs, |x|)$

Brute Force for LCS

BruteForceLCS(X, Y):

1. $lcs = 0$
2. **for** every subsequence x of X
3. **if** x is a subsequence of Y
4. $lcs = \textbf{Max}(lcs, |x|)$
5. return lcs

Brute Force for LCS

BruteForceLCS(X, Y):

1. $lcs = 0$
2. **for** every subsequence x of X
3. **if** x is a subsequence of Y
4. $lcs = \textbf{Max}(lcs, |x|)$
5. return lcs

Generating all the subsequences takes $O(2^{|x|})$ time.

Finding Optimal Substructure in LCS

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$
\begin{aligned}
X &= x_1\, x_2\, x_3\, \ldots\, x_m \\
Y &= y_1\, y_2\, y_3\, \cdots\, y_n
\end{aligned}
$$

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m = y_n$?

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m = y_n$?

Add x_m at the end of LCS of X and Y and find the

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m = y_n$?

Add x_m at the end of LCS of X and Y and find the LCS of $X[1 : m-1]$ and $Y[1 : n-1]$.

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m = y_n$?

Add x_m at the end of LCS of X and Y and find the LCS of $X[1 : m-1]$ and $Y[1 : n-1]$.

Is $LCS(i, j) = LCS(i-1, j-1) + 1$?

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$
\begin{aligned}
X &= x_1\, x_2\, x_3\, \ldots\, x_m \\
Y &= y_1\, y_2\, y_3\, \cdots\, y_n
\end{aligned}
$$

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$
\begin{aligned}
X &= x_1\ x_2\ x_3\ \ldots\ x_m \\
Y &= y_1\ y_2\ y_3\ \ldots\ y_n
\end{aligned}
$$

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m \neq y_n$?

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m \neq y_n$?

LCS of X and Y cannot end with both x_m and y_n.

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m \neq y_n$?

LCS of X and Y cannot end with both x_m and y_n.

So LCS of X and Y will be one among LCS of $X[1 : m]$ and $Y[1 : n - 1]$ and LCS of $X[1 : m - 1]$ and $Y[1 : n]$.

Finding Optimal Substructure in LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j)$ = length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

$$X = x_1\ x_2\ x_3\ \ldots\ x_m$$
$$Y = y_1\ y_2\ y_3\ \ldots\ y_n$$

What if $x_m \neq y_n$?

LCS of X and Y cannot end with both x_m and y_n.

So LCS of X and Y will be one among LCS of $X[1 : m]$ and $Y[1 : n-1]$ and LCS of $X[1 : m-1]$ and $Y[1 : n]$.

Is $LCS(i, j) = max(LCS(i-1, j), LCS(i, j-1))$?

Optimal Substructure in LCS

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then,

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \textbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then,

$Z = \underbrace{\mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m}$ will be an LCS of X and Y.

LCS of $x_1 x_2 \ldots x_{m-1}$ and $y_1 y_2 \ldots y_{n-1}$ followed by x_m

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \textbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof:

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \textbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1})$ is $k - 1$.

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1})$ is $k - 1$.

Suppose $W = w_1 w_2 \ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1})$ is $k - 1$.

Suppose $W = w_1 w_2 \ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Clearly, W ends with x_m.

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_{n-1})$ is $k - 1$.

Suppose $W = w_1 w_2 \ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Clearly, W ends with x_m. (Why?)

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is $k-1$.

Suppose $W = w_1w_2\ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Clearly, W ends with x_m. (Why?)

But then $w_1w_2\ldots w_{l-1}$ will be a **CS** of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_{n-1}$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is $k-1$.

Suppose $W = w_1w_2\ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Clearly, W ends with x_m. (Why?)

But then $w_1w_2\ldots w_{l-1}$ will be a **CS** of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_{n-1}$.

This is a contradiction as $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is shorter than $w_1w_2\ldots w_{l-1}$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is $k-1$.

Suppose $W = w_1w_2\ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Clearly, W ends with x_m. (Why?)

But then $w_1w_2\ldots w_{l-1}$ will be a **CS** of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_{n-1}$.

This is a contradiction as $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is shorter than $w_1w_2\ldots w_{l-1}$.

length $k-1$

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is $k-1$.

Suppose $W = w_1w_2\ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Clearly, W ends with x_m. (Why?)

But then $w_1w_2\ldots w_{l-1}$ will be a **CS** of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_{n-1}$.

This is a contradiction as $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is shorter than $w_1w_2\ldots w_{l-1}$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m = y_n$. Then, $Z = \mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1}) + x_m$ will be an LCS of X and Y.

Proof: Suppose length of Z is k and, hence, length of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is $k-1$.

Suppose $W = w_1w_2\ldots w_l$, not Z, is an LCS of X and Y, and $l > k$.

Clearly, W ends with x_m. (Why?)

But then $w_1w_2\ldots w_{l-1}$ will be a **CS** of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_{n-1}$.

This is a contradiction as $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_{n-1})$ is shorter than $w_1w_2\ldots w_{l-1}$.

length $k-1$

length $l-1$

■

Optimal Substructure in LCS

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least

Optimal Substructure in LCS

Claim: Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1 x_2 \ldots x_{m-1}, y_1 y_2 \ldots y_n)$ and $\mathbf{LCS}(x_1 x_2 \ldots x_m, y_1 y_2 \ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof:

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

If W doesn't end with x_m (WLOG), then W will be a $\mathbf{CS}$ of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_n$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

If W doesn't end with x_m (WLOG), then W will be a $\mathbf{CS}$ of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_n$.

This is a contradiction as W is longer than $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

If W doesn't end with x_m (WLOG), then W will be a $\mathbf{CS}$ of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_n$.

This is a contradiction as W is longer than $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

If W doesn't end with x_m (WLOG), then W will be a $\mathbf{CS}$ of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_n$.

This is a contradiction as W is longer than $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$.

length l

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

If W doesn't end with x_m (WLOG), then W will be a $\mathbf{CS}$ of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_n$.

This is a contradiction as W is longer than $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$.

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

If W doesn't end with x_m (WLOG), then W will be a $\mathbf{CS}$ of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_n$.

This is a contradiction as W is longer than $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$.

length l

length j

Optimal Substructure in LCS

Claim: Let $X = x_1x_2\ldots x_m$ and $Y = y_1y_2\ldots y_n$ be sequences such that $x_m \neq y_n$. Then, at least one out of $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ and $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ will be an $\mathbf{LCS}(X, Y)$.

Proof: Suppose neither $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ nor $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ is an $\mathbf{LCS}(X, Y)$.

Let $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$ be of length j.

Let $\mathbf{LCS}(x_1x_2\ldots x_m, y_1y_2\ldots y_{n-1})$ be of length k.

Let W be an $\mathbf{LCS}(X, Y)$ of length $l > \mathbf{Max}(j, k)$.

Since $x_m \neq y_n$, W cannot end with both x_m and y_n

If W doesn't end with x_m (WLOG), then W will be a $\mathbf{CS}$ of $x_1x_2\ldots x_{m-1}$ and $y_1y_2\ldots y_n$.

This is a contradiction as W is longer than $\mathbf{LCS}(x_1x_2\ldots x_{m-1}, y_1y_2\ldots y_n)$.

length l

length j

■

Recurrence for LCS

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$.

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$. Then,

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$. Then,

$$LCS(i, j) \; =$$

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$. Then,

$$LCS(i, j) = \begin{cases} \end{cases}$$

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$. Then,

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \end{cases}$$

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$. Then,

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1, j-1) + 1, & \text{if } x_i = y_j \end{cases}$$

Recurrence for LCS

Let $X = x_1 x_2 \ldots x_m$ and $Y = y_1 y_2 \ldots y_n$ be sequences.

Let $LCS(i, j) =$ length of the LCS of $x_1 x_2 \ldots x_i$ and $y_1 y_2 \ldots y_j$. Then,

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1, j-1) + 1, & \text{if } x_i = y_j \\ LCS(i,j) = max(LCS(i-1,j), LCS(i,j-1)), & \text{if } x_i \neq y_j \end{cases}$$

Overlapping Subproblems in LCS

Overlapping Subproblems in LCS

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1,j-1) + 1, & \text{if } x_i = y_j \\ LCS(i,j) = max(LCS(i-1,j), LCS(i,j-1)), & \text{if } x_i \neq y_j \end{cases}$$

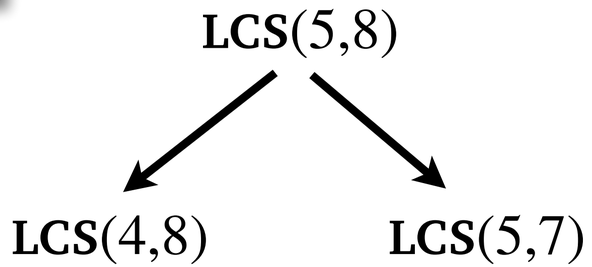
Overlapping Subproblems in LCS

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1,j-1) + 1, & \text{if } x_i = y_j \\ LCS(i,j) = max(LCS(i-1,j), LCS(i,j-1)), & \text{if } x_i \neq y_j \end{cases}$$

LCS(5,8)

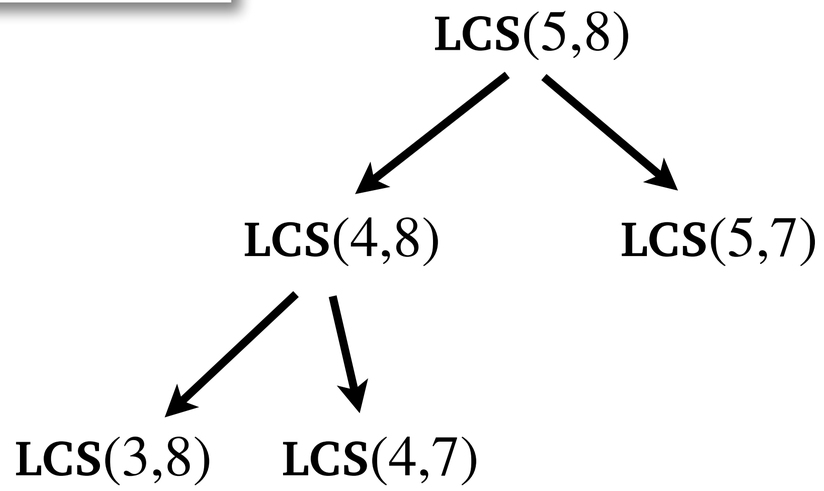
Overlapping Subproblems in LCS

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1,j-1) + 1, & \text{if } x_i = y_j \\ LCS(i,j) = max(LCS(i-1,j), LCS(i,j-1)), & \text{if } x_i \neq y_j \end{cases}$$



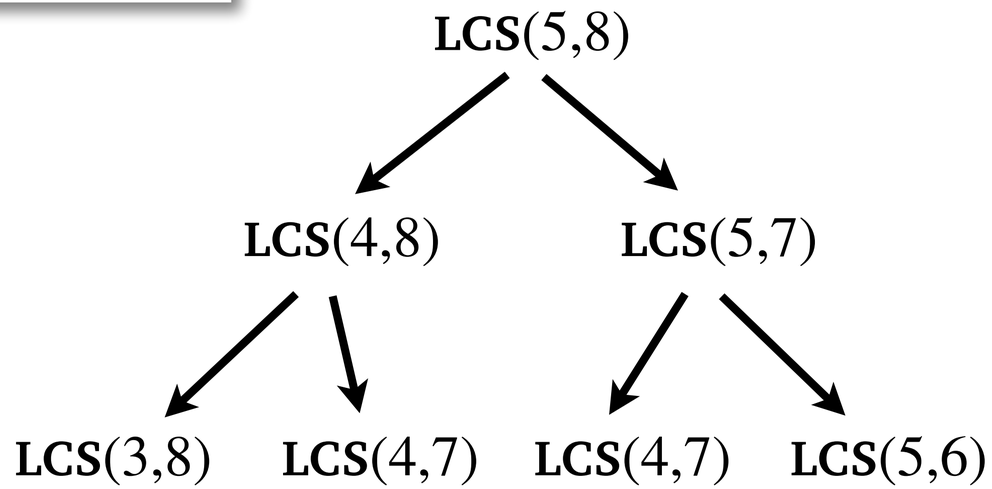
Overlapping Subproblems in LCS

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1,j-1) + 1, & \text{if } x_i = y_j \\ LCS(i,j) = max(LCS(i-1,j), LCS(i,j-1)), & \text{if } x_i \neq y_j \end{cases}$$



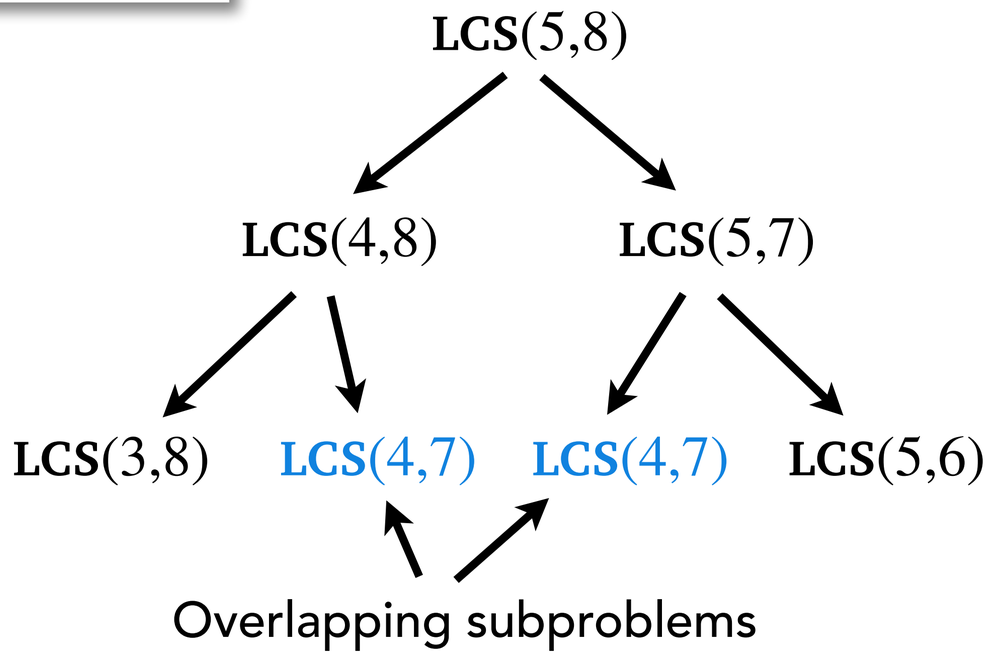
Overlapping Subproblems in LCS

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1,j-1) + 1, & \text{if } x_i = y_j \\ LCS(i,j) = max(LCS(i-1,j), LCS(i,j-1)), & \text{if } x_i \neq y_j \end{cases}$$



Overlapping Subproblems in LCS

$$LCS(i,j) = \begin{cases} LCS(i,j) = 0, & \text{if } i = 0 \text{ or } j = 0 \\ LCS(i,j) = LCS(i-1,j-1) + 1, & \text{if } x_i = y_j \\ LCS(i,j) = max(LCS(i-1,j), LCS(i,j-1)), & \text{if } x_i \neq y_j \end{cases}$$



Top-Down DP for LCS

Top-Down DP for LCS

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$
2. **return** $dp[i][j]$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$
2. **return** $dp[i][j]$
3. **if** $x_i == y_j$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$
2. **return** $dp[i][j]$
3. **if** $x_i == y_j$
4. $dp[i][j] =$ **LCS**$(i-1, j-1) + 1$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$
2. **return** $dp[i][j]$
3. **if** $x_i == y_j$
4. $dp[i][j] =$ **LCS**$(i-1, j-1) + 1$
5. **else**

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$
2. **return** $dp[i][j]$
3. **if** $x_i == y_j$
4. $dp[i][j] =$ **LCS**$(i-1, j-1) + 1$
5. **else**
6. $dp[i][j] =$ **Max**(**LCS**$(i-1, j)$, **LCS**$(i, j-1)$)

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$
2. **return** $dp[i][j]$
3. **if** $x_i == y_j$
4. $dp[i][j] =$ **LCS**$(i-1, j-1) + 1$
5. **else**
6. $dp[i][j] =$ **Max**(**LCS**$(i-1, j)$, **LCS**$(i, j-1)$)
7. **return** $dp[i][j]$

Top-Down DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

$dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

$dp[0][0] = 0$

LCS(i, j): *// Call with* ***LCS***$(|X|, |Y|)$

1. **if** $(dp[i][j] \neq -1)$
2. **return** $dp[i][j]$
3. **if** $x_i == y_j$
4. $dp[i][j] =$ **LCS**$(i-1, j-1) + 1$
5. **else**
6. $dp[i][j] =$ **Max**(**LCS**$(i-1, j)$, **LCS**$(i, j-1)$)
7. **return** $dp[i][j]$

Time Complexity: $O(nm)$

Bottom-Up DP for LCS

Bottom-Up DP for LCS

LCS(X, Y)

Bottom-Up DP for LCS

LCS(X, Y)

1. $dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0:n][0:m] = \{-1, -1, \ldots, -1\}$

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i - 1][j - 1] + 1$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0						
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

0	0	0	0	0	0	0
0						
0						
0				(i, j)		
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

dp

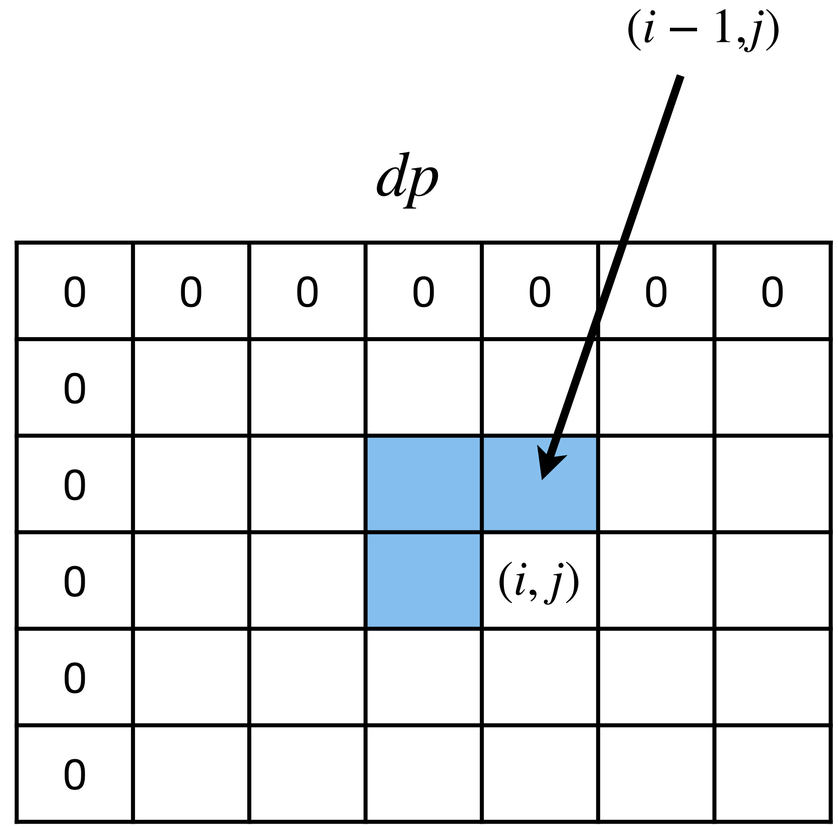
0	0	0	0	0	0	0
0						
0						
0				(i, j)		
0						
0						

Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

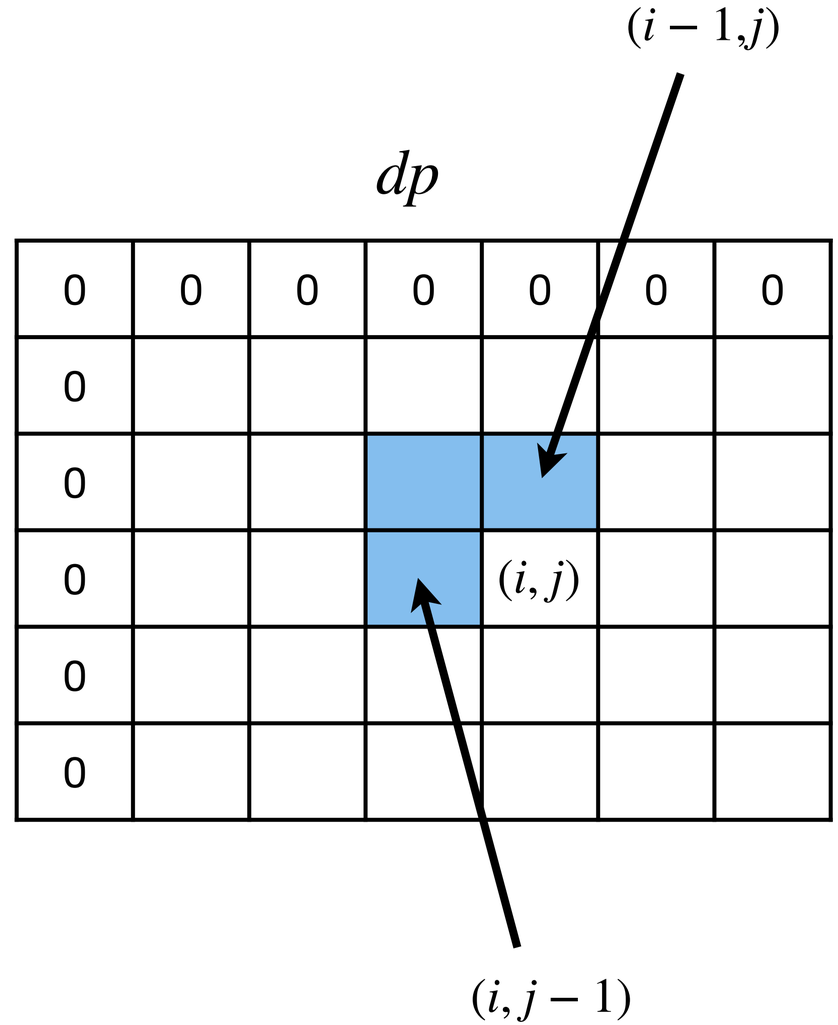


Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

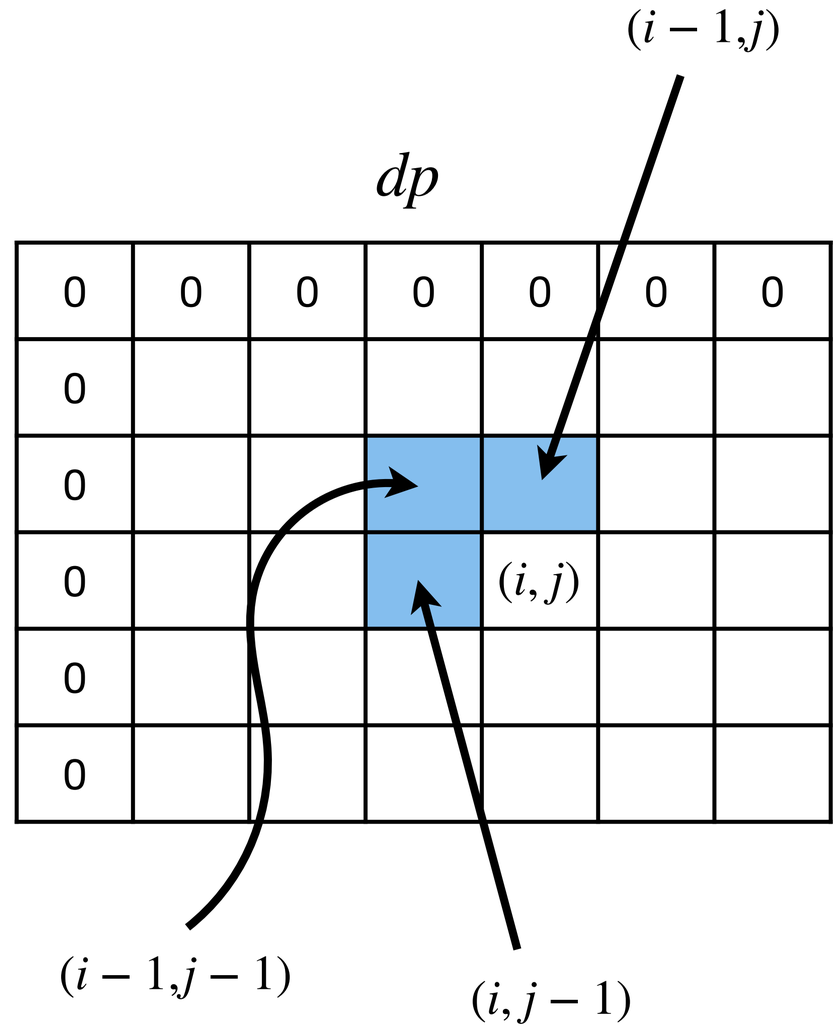


Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$



Bottom-Up DP for LCS

$dp[i][j]$ stores $LCS(i, j)$

LCS(X, Y)

1. $dp[0 : n][0 : m] = \{-1, -1, \ldots, -1\}$
2. $dp[0][0 : m] = 0, dp[0 : n][0] = 0.$
3. **for** $i = 1$ **to** n
4. **for** $j = 1$ **to** m
5. **if** $x_i == y_j$
6. $dp[i][j] = dp[i-1][j-1] + 1$
7. **else**
8. $dp[i][j] = \textbf{Max}(dp[i-1][j], dp[i][j-1])$
9. **return** $dp[n][m]$

Time Complexity: $O(nm)$

Bottom-Up vs Top-Down DP

Bottom-Up vs Top-Down DP

- Top-down solves only those subproblems which are necessary.

Bottom-Up vs Top-Down DP

- Top-down solves only those subproblems which are necessary.
- Bottom-up solves all.

Bottom-Up vs Top-Down DP

- Top-down solves only those subproblems which are necessary.
- Bottom-up solves all.
- If all subproblems need to be solved, then bottom up is better due to no recursion overhead

Bottom-Up vs Top-Down DP

- Top-down solves only those subproblems which are necessary.
- Bottom-up solves all.
- If all subproblems need to be solved, then bottom up is better due to no recursion overhead
- Otherwise, then top down is better.